

ADVANCED WEB DEVELOPMENT FRAMEWORKS (ITUE301)

Practical 2: State Management and Routing in React	
CO/PO	CO1 / PO3, PO5
Objective	To implement reactive state management using useState and multi-page navigation using React Router in the existing portfolio application.
Prerequisites	• Practical 1 completed working portfolio app with reusable components • Basic understanding of JavaScript array/object handling
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 2 IBM: Developing Front-End Apps with React, Module 2 (props, event handling, component composition, useState hook).
Architecture / Diagram	
Diagram	<pre> App.jsx (wrapped in <BrowserRouter>) ├── NavBar.jsx (links to all routes) ├── Route: "/" → Home.jsx ├── Route: "/projects" → Projects.jsx └── Route: "/contact" → Contact.jsx (controlled form, useState) </pre>
Problem Definition	
Problem Statement	• Extend the portfolio application from Practical 1 by adding React Router. • Implement a navigation bar with at least 3 routes: Home, Projects, and Contact. • Each route must render a distinct component without a full page reload. • Add at least 2 useState variables used meaningfully one for toggling UI visibility and one for managing a form input on the Contact page. • The Contact page must include a controlled input that captures and displays user input in real time.
Key Questions / Analysis	• What is the role of each component in the overall UI structure? • How does React re-render when props change? • Why is component reusability important in large-scale applications?
Supplementary Problems	• Add a 404 Not Found route that renders a custom error component for unknown paths. • Implement a dark/light mode toggle using useState that applies a CSS class to the root element. • Store the contact form input in state and display a live character count below the input.
Key Skills Addressed	• Installing and configuring React Router v6 • Defining and navigating between routes without page reload • Managing component state using the useState hook • Building controlled form inputs tied to state
Learning Outcome	Students will be able to implement client-side routing with React Router and manage dynamic UI state using useState, resulting in a multi-page application without full page reloads.
Lab Session Step-by-Step	
Steps	1. Install React Router in the existing portfolio project:

	<pre>npm install react-router-dom</pre> 2. Wrap the App component with BrowserRouter in main.jsx: <pre>import { BrowserRouter } from 'react-router-dom'; <BrowserRouter> <App /> </BrowserRouter></pre> 3. Define 3 routes inside App.jsx using Routes and Route: <pre><Routes> <Route path="/" element={<Home />} /> <Route path="/projects" element={<Projects />} /> <Route path="/contact" element={<Contact />} /> </Routes></pre> 4. Build the NavBar using Link (not <a> tags) to avoid full page reloads. 5. Inside Contact.jsx, create a controlled input tied to useState: <pre>const [message, setMessage] = useState(''); <input value={message} onChange={(e) => setMessage(e.target.value)} /></pre> 6. Add a second useState variable to toggle visibility of an element (e.g., a help tooltip). 7. Test navigation across all 3 routes and confirm no page reload occurs.	
Tools / Technology / Resources Required	• Node.js (v18+), npm • Visual Studio Code • React 18+, React Router v6 • Browser DevTools	
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours	
Faculty Teaching Guide		
Guidance Notes	• Emphasize Link vs <a> this is the most common point of confusion for students new to SPA routing. • Live-demo useState updating the UI in real time before students attempt the Contact form. • Check that BrowserRouter wraps the App only once, typically in main.jsx, not repeated elsewhere. • Walk through how routes map to components before students start writing code.	
Common Mistakes	• Using instead of <Link to="...">, causing full page reloads. • Forgetting to wrap the app in BrowserRouter, resulting in a 'useRoutes() may be used only in the context of a Router' error. • Using props instead of state for values that should change dynamically (like form input). • Not providing a unique path for each Route, causing routes to render incorrectly.	
Troubleshooting Guide		
Symptom	Likely Cause	Fix
Blank page after adding routes	BrowserRouter not wrapping App component	Confirm BrowserRouter wraps <App /> in main.jsx
Clicking nav link reloads page	Used <a> tag instead of <Link>	Replace all navigation anchors with <Link to="..."> from react-router-dom
Form input not updating on typing	Missing onChange handler or value not tied to state	Ensure input has both value={state} and onChange={set state}

Route shows wrong component	Path mismatch or routes defined in wrong order	Double check path strings match exactly; order routes from specific to general
Student Assignment / Post Laboratory Work		
Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a 404 Not Found route for any undefined path. • Implement a dark/light mode toggle using a second useState variable.	
GitHub Deliverables		
Expected Repository Contents	• Same repository as Practical 1, updated with new commits • Working multi-route app with NavBar, Home, Projects, Contact • At least one commit specifically for routing implementation • Updated README noting new routes added	
Rubrics (Practical Evaluation / Viva)		
Criteria (Marks)	Description	
Conceptual understanding (2 Marks)	Explains what and why, not just what was done	
Implementation (4 Marks)	Code written, runs, and is original	
Correctness (2 Marks)	Output matches expected result	
Reflection (1 marks)	Errors documented and explained	
Lab file evidence (1 marks)	Completeness and presentation	
Total: 10 marks	Passing Threshold: 5 / 10 (50%)	